PCP OS

# PCP OS Overview

What PCP OS is, what exists today, and the ideas it is built on. Plain language.

OVERVIEW

ENTERPRISE EDITION

**Kirtus Justus**

Creator and owner

Parrot Council Protocol (PCP) 1.0 · PCP OS, build 19 · 12 November 2026

Early development. Untested and not peer reviewed. Started only in an emulator. See EVIDENCE_STATUS.md.

Evidence before agreement.

# PCP OS Overview

**Product:** Parrot Council Protocol (PCP), Version 1.0. **Component:** PCP OS, build 19. **Method carried:** Adversarial Claim Review (ACR) 1.0. **Creator:** Kirtus Justus. **Date:** 12 November 2026. **This document explains. It requires nothing.** The requirements are in the PCP OS Specification.

## What PCP OS is

PCP OS is an operating system being built around one idea: the record of what was examined, by whom and on what evidence is kept by a mechanism that the examined party cannot rewrite.

It is part of Parrot Council Protocol, the product. The method it carries is Adversarial Claim Review, which says how a claim is examined and what its evidence is permitted to establish.

It is early. Read the next section before anything else.

## What exists today

Two things exist and were run for this build.

**A kernel.** New, written in Rust, for 64-bit Intel and AMD processors, sharing no code with any other kernel. In an emulator it starts, protects its own memory, proves the protection by attempting each forbidden access and being refused by the processor, takes over exceptions and a timer, runs thirteen small programs in user mode as processes, each in its own address space, takes the processor back from the ones that do not give it up, ends each one that does something the processor refuses, keeps a register of its own start, its processes and its stop, prints 54 self-test results, and stops. The programs are built into the kernel image: it cannot load one from anywhere else. It has no storage, no network, no display and no users. It has never been started on a real computer.

**A program for other systems.** The same core, built as an ordinary program called `pcpos` for Linux. It keeps the register, checks every event against the method's mechanical rules, serves assistants through a standard tool interface, and records what the host itself observes. This is what the `pcp-os` plugin uses. It is as trustworthy as the system it runs on, and no more.

Everything else in this document is design.

## The idea that shapes everything: two paths

A computer exists to do your work. A method for examining claims must not turn the processor into a debating chamber.

So PCP OS keeps two paths apart. On the **work path**, programs compute, read, write and draw, and no rule of ACR is ever consulted. On the **record path**, someone offers an event to a register, such as "this check ran" or "this finding was reached", and a guard looks at that one event. A guard is a necessary condition that can be checked without judgment: gates in order, a finding citing something that is in the record, nothing after a halt.

When ACR is not in use, the system does nothing about ACR at all.

This was measured, and one measurement found a fault. Recording an event used to take longer the more events a register held: 21 milliseconds each at two thousand events. It now takes about a quarter of a millisecond at any size tried, stored durably.

## Who is in charge

You are. The design has no all-powerful account. Authority is held as capabilities, which are specific rights that can be passed on only in the same or a narrower form and can be taken back with everything derived from them.

An assistant or a program can never hold the right to start ACR, to administer the system or to change it. That is checked exhaustively in the core, for every combination of roles.

Administration is something a person does, in a state of the system called maintenance, and each act is to be recorded before it takes effect.

## What you may change

Everything, in the end. It is your machine and your copy.

What the design insists on is that a change is never silent. The method, each module, each program and the kernel image are identified by digests, and every record says which digests it was made under. If you change a module, it is a different module with its own digest, and reviews made with it say so. If you rebuild the kernel, its measurement changes, and later records carry the new one. Nobody is stopped. Nobody is fooled either.

Two things do not change in place: a review keeps the method and modules it started with until it closes, and a module that is not a released module does not carry a released module's name.

## Modules and workspaces

Core applies whenever ACR is in use. Mathematics, Science, Engineering, Cryptography, Statistics and Quality Assurance are specialist modules, and apply only when you choose them.

A workspace is a project's register. You can tell it which modules its reviews use, for instance a science workspace. The assistant cannot choose for you. Each module has its own colours, so a science workspace looks like one; the colours repeat what the words say and decide nothing.

A module is found by reading the heading of its own text. Adding one is adding a folder.

## Languages, and why there is no rule against Python

Rust comes first: the kernel and every system service are written in it, and where two implementations disagree the Rust one is right.

C and C++ are for what already exists in them: device code, vendor drivers, existing programs. They are to run only inside compartments that hold exactly the rights they were given, so that their memory errors are contained instead of hoped away.

Python and similar languages are treated as what they are: an interpreter is one more contained program, and a script is data handed to it.

None of this is written as a test of which language something is. It is a matter of where code may run and what it may hold. The system's own services are given no way to reach an interpreter, so the system's own checks cannot depend on one. The `pcp-os` plugin follows the same rule today: its program is Rust and uses no interpreter, including for looking up a clause.

The Python implementation of the core exists because most people can run Python. It is tested against 17,113 cases the Rust implementation publishes. It is a convenience, not an authority.

## Extending it

PCP OS is meant to be extended by the person who holds it, with other people's extensions and their own. The design takes its lesson from where other systems were hurt: an extension gets a manifest saying what it is and what it asks for, receives only the rights you grant, never runs inside the kernel, and can have its rights taken back.

## Display, time and first start

Designed, not built. The display is to be used at its own resolution from the first frame. Backgrounds are drawn from shapes, not stretched from pictures, in the colours of your edition and workspace; those files exist now. Records are always in universal time; your time zone is your own setting, asked once, and needs no network. First start creates you, the person who directs the system, before anything else, and asks for no account anywhere.

## What it does not protect against

On another operating system, anything running with your rights can write the register file directly. The record makes that detectable only if you kept its head digest somewhere else. The kernel is what is meant to close this, and the kernel cannot yet run a program.

The kernel itself has no address randomization, no signed image, no defence against attacks on speculative execution, and no protection against someone with the machine in their hands.

A halt stops ACR use. It is not a safe state for anything physical.

## Status

Not peer reviewed, not independently reviewed, not evaluated in use. Nothing here is certified. Proprietary; all rights reserved. Copyright (c) 2026 Kirtus Justus.